A Dynamic DDoS Traffic Scrubbing System Based on a Programmable Data Plane

English translation — September 2026

Translated from the original Chinese competition report. All sections, formulas, algorithms, experimental results, and references are retained. Figure labels are translated; the original diagrams and plotted data are preserved. Lettered notes identify inconsistencies in the source without silently changing its claims.

Chinese original (PDF) — Project repository

Abstract

Link-flooding DDoS attacks use traffic flooding to disconnect a target network from the Internet, causing widespread network outages and posing a serious threat to Internet security. Traffic scrubbing is a common defense against link-flooding DDoS attacks. However, existing traffic scrubbing methods often use static scrubbing rules, making it difficult to address dynamic link-flooding DDoS attacks. To address this problem, this report proposes a dynamic traffic scrubbing method for link-flooding DDoS attacks based on the programmable data plane, termed the Dynamic DDoS Traffic Scrubbing System (DDSS). The method introduces moving target defense (MTD) into traffic scrubbing and periodically changes the deployment of scrubbing rules to prevent attackers from launching effective attacks. It also uses game theory to analyze the DDoS attack–defense game. Experiments show that the proposed DDSS effectively withstands dynamic link-flooding DDoS attacks while introducing little overhead.

Keywords: traffic scrubbing; moving target defense; link-flooding DDoS; P4; programmable data plane

1 Introduction

The cybersecurity landscape is complex and constantly changing, with security threats present everywhere. The outbreak of COVID-19 accelerated the development of the "contactless economy" and increased people's dependence on digital and network technologies. As a result, cyberspace has come to accommodate an increasing share of national interests, further intensifying cybersecurity risks. Distributed denial-of-service (DDoS) attacks, such as Coremelt and Crossfire, are a typical form of network attack. By flooding critical network links, they can disconnect a target network from the Internet and cause widespread network outages. These attacks pose an extremely serious threat to cyberspace and can even threaten national security. Between 2013 and 2022, the Internet experienced multiple large-scale link-flooding DDoS attacks, resulting in numerous Internet service disruptions and incalculable losses[1].

The principal defenses against link-flooding DDoS attacks fall into three categories: attack traffic detection, rerouting, and traffic scrubbing. Traffic scrubbing directly drops attack traffic at the network edge or within the network, effectively reducing its impact. However, traditional defenses against link-flooding DDoS attacks depend on prior knowledge of attacks and are severely limited by delayed responses and static operation. Attackers can readily identify and bypass them, making it difficult for these defenses to respond effectively to organized, high-intensity link-flooding DDoS attacks. Moving target defense (MTD) is a new cybersecurity concept[2] that uses proactive mechanisms involving dynamics, randomness, and diversity to limit the exposure of network vulnerabilities. By increasing the complexity and cost of network attacks, it can substantially reduce their destructive impact.

To address these problems, this report introduces MTD into traffic scrubbing, dynamically scrubbing attack traffic in a network with a programmable data plane so that attackers have difficulty mounting effective attacks. First, a traffic scrubbing architecture for link-flooding DDoS attacks is

constructed using the P4 (Programming Protocol-Independent Packet Processors) language[3] and virtual network functions (VNFs)[4], employing the programmable data plane to scrub DDoS attack traffic. Second, an MTD-based dynamic traffic scrubbing method is proposed. It periodically changes the traffic selected for scrubbing and deploys the corresponding scrubbing rules, preventing attackers from launching targeted attacks. Third, game theory is used to analyze the DDoS attack–defense game, and a Nash equilibrium solution is provided. Finally, a dynamic traffic scrubbing system is implemented. Experiments show that the proposed method effectively addresses link-flooding DDoS attacks while introducing little overhead.

2 Related Work

2.1 Typical Link-Flooding DDoS Attack Methods

Two typical link-flooding DDoS attacks are the Coremelt attack[5] and the Crossfire attack[6]. The two approaches follow similar attack strategies: both use bots to send attack traffic and congest target links. The Coremelt attack (left side of Figure 1) sends attack traffic between bots, whereas the Crossfire attack (right side of Figure 1) uses bots to send attack traffic to decoy servers and target servers. By disguising attack traffic as legitimate traffic, Crossfire achieves greater stealth.

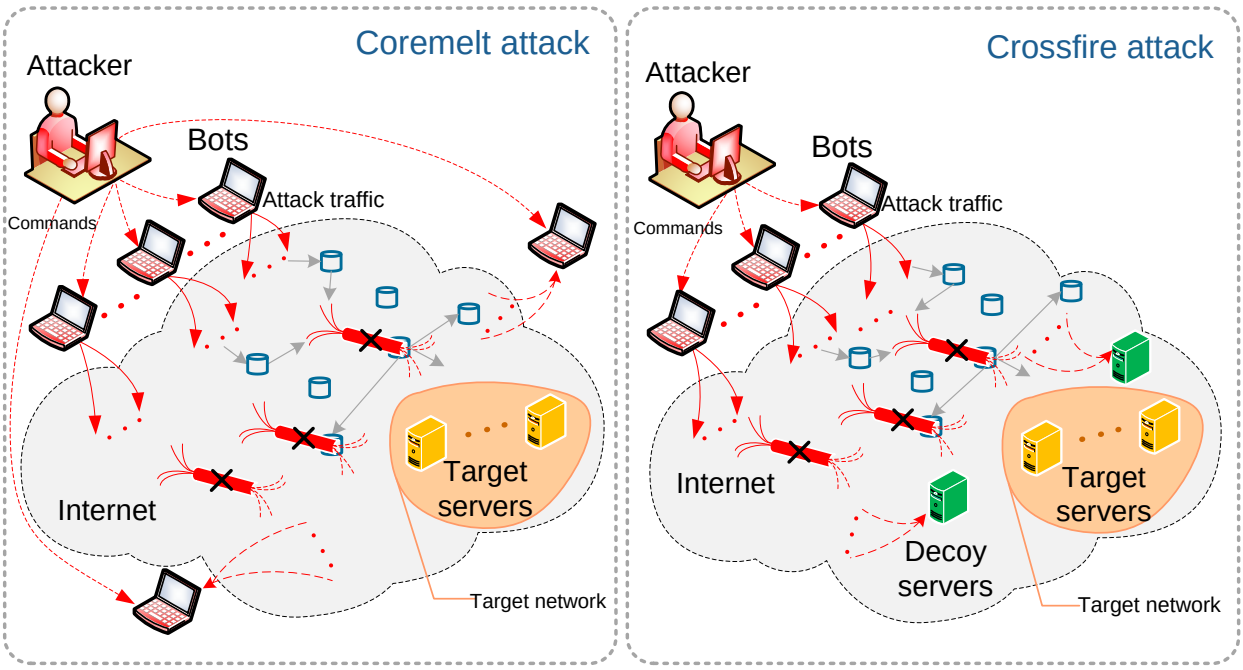


Figure 1: Illustration of Coremelt and Crossfire attacks.

The lifecycle of a link-flooding DDoS attack can be divided into three phases:

1. **Attack preparation:** The attacker sends probe packets into the network through bots to obtain the network topology and uses stable routes to identify target links.
2. **Attack launch:** The attacker coordinates bots to send attack traffic and congest the target links.
3. **Attack adjustment:** The attacker adjusts the attack traffic sent by bots to counter the defender and prolong the attack.

2.2 Defenses Against Link-Flooding DDoS Attacks

Traditional defenses against link-flooding DDoS attacks respond passively after an attack has occurred. They primarily comprise three typical methods: attack detection, traffic rerouting, and traffic scrubbing.

(1) Attack detection methods. Link-flooding DDoS detection is a typical approach to resisting link-flooding DDoS attacks. It uses traffic characteristics and network conditions to determine whether the network is under attack or to locate the attack. Link-flooding DDoS traffic closely resembles legitimate traffic and is highly covert. However, to sustain the impact of an attack, attackers adjust their strategies in response to defensive measures, exhibiting behavior that differs from legitimate traffic. Exploiting this difference, Gkounis et al.[7] proposed a traffic-engineering-based method for detecting link-flooding DDoS attacks. Based on a software-defined networking (SDN) architecture, the method repeatedly changes traffic routes and detects attack traffic that responds persistently. Similarly, Liaskos et al.[8] used rerouting to force attackers to adjust their strategies and thereby reveal their malicious behavior, and modeled attack detection using relational algebra. Given the enormous number of Internet links, defenders cannot monitor all of them. Ma et al.[9] therefore randomly monitored a subset of network nodes within each detection period. By continuously changing the monitored node set, the approach makes attackers perceive that detection is being performed throughout the network.

The preceding methods all require experts to provide detection models, resulting in high labor costs. To address this issue, Chen et al.[10] proposed a convolutional-neural-network-based method for detecting link-flooding DDoS attacks. The method constructs a training set from consecutive temporal snapshots of network transmission states. It uses the automatic feature-learning capability of convolutional neural networks to train an attack detection model end to end, avoiding manual intervention and providing objective, quantitative assessments of security risks.

Precise defense against link-flooding DDoS attacks requires identifying the attack targets or the locations of bots so that traffic can be rerouted or filtered selectively. To this end, Xue et al.[11] used end-to-end and hop-by-hop network measurements to detect anomalous degradation in path performance. By correlating performance data with probe packet data, they inferred the links or regions under attack. Xing et al.[12] identified bots by classifying the source IP addresses of traffic contributing to congestion, thereby cutting off the sources of attack traffic. Their method used an SDN network to collect periodic traffic statistics on congested links and a deep convolutional neural network to identify bots efficiently and accurately.

These attack detection methods attempt to detect link-flooding DDoS attacks after they have already affected the network, making the defense reactive and delayed. Meanwhile, as link-flooding DDoS attacks continue to evolve, their traffic becomes increasingly covert, substantially increasing the difficulty and overhead of detection.

(2) Traffic rerouting. Traffic rerouting mitigates link-flooding DDoS attacks by scheduling traffic to change its distribution across the network. The centralized and efficient network management and control provided by SDN facilitate timely, accurate traffic measurement and dynamic, flexible traffic scheduling. However, existing commercial networks cannot be replaced entirely with SDN networks. Wang et al.[13] therefore selected only a subset of network nodes for upgrading to SDN nodes. When a link-flooding DDoS attack is detected, these SDN nodes balance traffic across all network links to relieve congestion. Xie et al.[14] used a more flexible segment routing mechanism, selecting traffic according to its level of suspicion and routing it around congested links to maintain network connectivity.

The preceding methods design rerouting schemes from a local network perspective to mitigate congestion. In contrast, Lee et al.[15] proposed a defense mechanism based on cooperation among autonomous systems to protect legitimate traffic. The method coordinates multiple autonomous

systems that have not been compromised by bots, reroutes traffic from congested links over multiple paths, and limits the bandwidth of attack traffic on congested paths. Ding et al.[16] proposed link decoys to mitigate link-flooding DDoS attacks in the Internet of Things by providing adversaries with false link maps. The method selectively reroutes probe flows and extracts distinctive traffic features from the link-map construction and flooding phases, thereby concealing target links from attackers.

When responding to link-flooding DDoS attacks, these methods reroute some traffic to balance load or divert suspicious traffic. However, they select traffic on congested links or rerouting paths in isolation, making it difficult to obtain an optimal traffic rerouting scheme. Furthermore, none of these methods considers traffic requirements for network functions. After rerouting, traffic may therefore no longer receive the services provided by those functions.

(3) Traffic scrubbing. Traffic scrubbing identifies and intercepts attack traffic according to configured scrubbing rules and can eliminate the impact of link-flooding DDoS attacks on the network. Scrubbing rules can be deployed at the network edge or within the network. Exploiting the elastic deployment of network functions enabled by network function virtualization (NFV), Fayaz et al.[17] dynamically deployed software traffic scrubbing engines at the network edge according to the volume and type of attack traffic, then redirected suspicious traffic to those engines to mitigate network congestion. Zhang et al.[18] used the customizable packet-processing logic and high-speed packet-processing capabilities of programmable switches to transform traffic scrubbing logic into a match-action model. They constructed a traffic scrubbing center using programmable switches and server software to provide efficient traffic scrubbing.

In contrast to these approaches, Zhang et al.[19] proposed scrubbing traffic within the network by deploying scrubbing rules across multiple autonomous systems. Removing attack traffic as early as possible reduces the overall network load and the probability of link congestion. Building on this approach, Sisodia et al.[20] further investigated the selection of traffic scrubbing rules. Taking network resource constraints into account, they proposed a scrubbing-rule selection method based on an offer model. This method grants autonomy to networks: in response to a defense request, a network independently proposes several sets of scrubbing rules, and the user makes the final selection through optimization.

These traffic scrubbing methods process attack traffic either at the network edge or within the network. The former requires attack traffic to be routed to a traffic scrubbing center, wasting network bandwidth. Although the latter does not consume bandwidth resources, high-intensity attacks require it to occupy substantial flow-table resources, affecting data forwarding speed.

3 Method

Defending against high-intensity link-flooding DDoS attacks requires the deployment of many scrubbing rules. However, limited switch flow-table space means that defenders can scrub only a subset of attack traffic. Attackers can therefore adjust their traffic to evade scrubbing and prolong the attack. If the defender continually and randomly changes the categories of traffic being scrubbed, it can prevent attackers from making targeted adjustments even when flow-table space is limited, thereby preventing them from achieving the intended attack effect. Section 3.1 describes the DDSS framework. Section 3.2 combines P4 switches and VNFs to construct a traffic scrubbing engine. Section 3.3 analyzes the DDoS attack–defense game and presents an algorithm for computing its Nash equilibrium.

3.1 DDSS Framework

The proposed DDSS framework is shown in Figure 2. Every node in the network is a programmable switch connected to a dedicated traffic scrubbing server. The attacker uses a collection of bots under its control to send large volumes of attack traffic toward target links, congesting those links. According to the categories of attack traffic, the defender configures the programmable switches, installs traffic scrubbing VNFs on the scrubbing servers, and dynamically deploys traffic scrubbing rules in the network. Programmable switches and VNFs then work together to scrub attack traffic. The attacker periodically adjusts the attack traffic to evade scrubbing. Assuming that each party knows the other party's information and that neither acts before the other, link-flooding DDoS attack and defense constitute a static game of complete information. The defender can use the game equilibrium to construct its optimal scrubbing strategy.

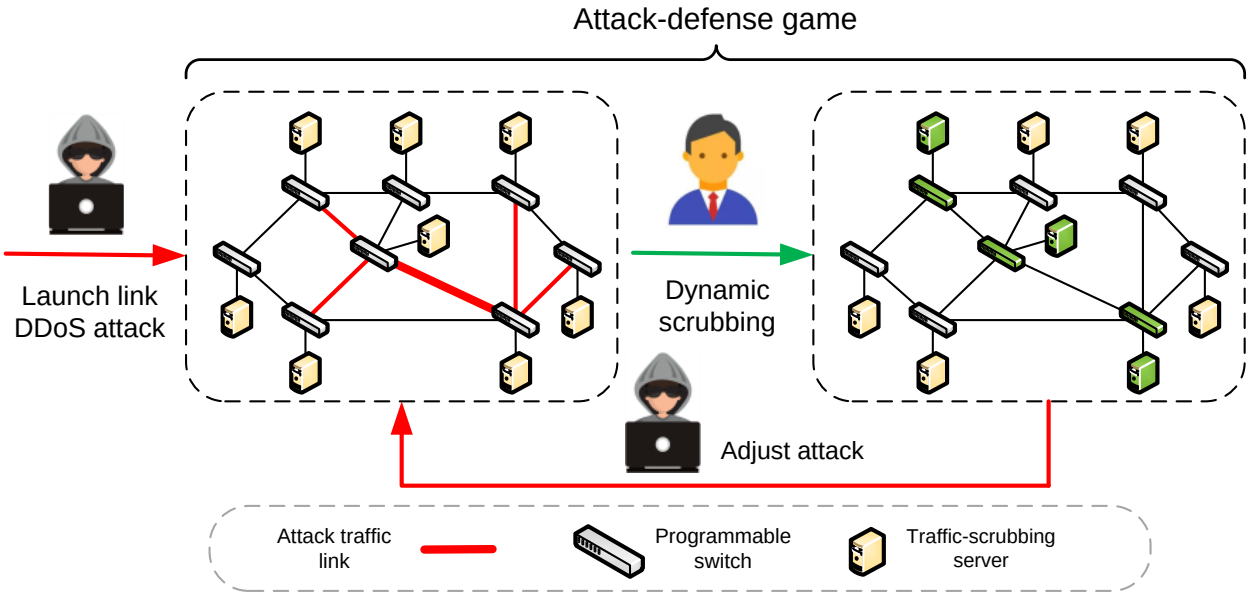


Figure 2: The DDSS framework.

3.2 Traffic Scrubbing Engine Based on P4 Switches and VNFs

P4 switches provide high packet-processing speed and throughput and support stateful packet processing, enabling high-speed traffic scrubbing within the network. However, their supported packet-processing logic is limited: they can scrub only certain categories of attack traffic using the match-action model. This report therefore combines P4 switches and VNFs to construct a traffic scrubbing engine that can be deployed elastically across the network in a distributed manner.

As shown in Figure 3, each P4 switch in DDSS is connected to a dedicated traffic scrubbing server. The server can dynamically deploy traffic scrubbing VNFs, cooperating with the P4 switch to scrub traffic. Both components are centrally managed by the SDN controller and together form a traffic scrubbing engine. The SDN controller configures the engine according to network conditions and the defense strategy to scrub attack traffic.

Traffic scrubbing rules for link-flooding DDoS attacks can be divided into two categories according to whether they can be converted to the match-action model[18]. Scrubbing logic that can be expressed in this model is equivalently translated into packet-processing logic supported by the switch using the P4 data-plane programming language and deployed in the data plane. Scrubbing logic that cannot be implemented using the match-action model is implemented in software and deployed elastically in the network as VNFs, which work with programmable switches to scrub traffic. To reduce the load on switches and traffic scrubbing servers, the SDN controller deploys

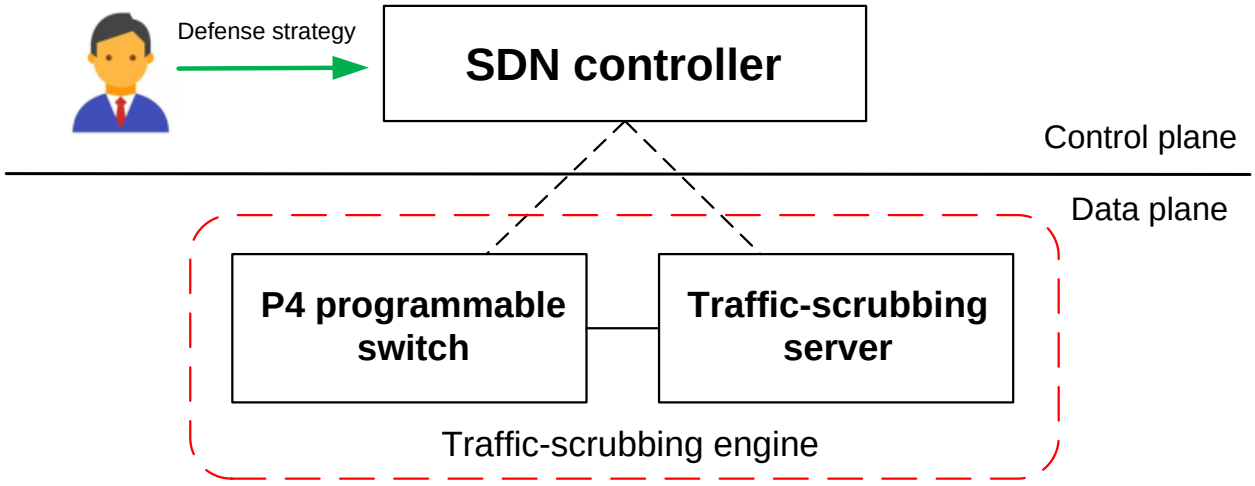


Figure 3: The traffic scrubbing engine.

scrubbing rules and VNFs at multiple network nodes according to the available flow-table space of switches along attack traffic paths, thereby balancing the load among switches.

3.3 The Link-Flooding DDoS Attack–Defense Game

In the link-flooding DDoS attack–defense game, the attacker launches attacks and continually adjusts the attack traffic, while the defender selects some of that traffic for scrubbing to prevent the attack from having its intended effect. Each party's benefit is affected by the other party's strategy, and both take actions that favor themselves and maximize their own benefit. Assuming that both parties know each other's information and act without a sequential order, this process can be modeled as a normal-form game with complete information. According to game theory, a Nash equilibrium is an equilibrium solution of this game: neither party can obtain a higher payoff by unilaterally deviating from it. The Nash equilibrium therefore gives the defender's optimal scrubbing strategy. Based on this analysis, we formulate the Traffic Scrubbing Game (TSG).

Definition 1. TSG is a four-tuple, $\mathrm{TSG} = \{N, S, D, U\}$, where:

1. $N = \{A, D\}$ is the set of players, where A is the attacker and D is the defender.
2. $S^a = \{s_0^a, s_1^a, s_2^a, \ldots, s_g^a\}$ is the attacker's strategy space. For $s^a \in S^a$, $s^a \in \mathbb{Z}$ and $0 \le s^a \le g$; it denotes the number of attack flows selected by the attacker.
3. $S^d = \{s_0^a, s_1^d, s_2^d, \ldots, s_g^d\}$ is the defender's strategy space. For $s^d \in S^d$, $s^d \in \mathbb{Z}$ and $0 \le s^d \le g$; it denotes the number of attack flows that the defender can select for scrubbing.
4. $U = \{U^a, U^d\}$ is the set of payoff functions of the players. U^a and U^d denote the attacker's and defender's payoff functions, respectively, and can be written as

$$U^a = \begin{bmatrix} u^a(s_0^a, s_0^d) & \cdots & u^a(s_0^a, s_g^d) \\ \vdots & \ddots & \vdots \\ u^a(s_g^a, s_0^d) & \cdots & u^a(s_g^a, s_g^d) \end{bmatrix}, \qquad U^d = \begin{bmatrix} u^d(s_0^a, s_0^d) & \cdots & u^d(s_0^a, s_g^d) \\ \vdots & \ddots & \vdots \\ u^d(s_g^a, s_0^d) & \cdots & u^d(s_g^a, s_g^d) \end{bmatrix}.$$

   Here, $u^a(s_i^a, s_j^d)$ and $u^d(s_i^a, s_j^d)$, with $i \in [0, g]$ and $j \in [0, g]$, denote the respective payoffs when the attacker selects strategy s_i^a and the defender selects strategy s_j^d.

Translation note[a].

In the link-flooding DDoS attack–defense game, let attack payoff equal attack benefit minus attack

cost, and defense payoff equal defense benefit minus defense cost. To obtain the attack and defense payoff matrices, we define attack benefit, attack cost, defense benefit, and defense cost.

Definition 2. The attack benefit $E^a(n^a, n^d)$ is the expected benefit generated by attack flows that are not scrubbed by the defender when the attacker selects n^a attack flows and the defender selects n^d attack flows for scrubbing. The attack benefit is linear in the number of attack flows not scrubbed by the defender. Let h_k denote the number of flows against which the defense is unsuccessful. Then

$$E^a(n^a, n^d) = \lambda h_k, \qquad 0 \le n^a \le g, \quad 0 \le n^d \le g.$$

Definition 3. The attack cost $AC(n^a)$ is the cost incurred when the attacker selects n^a attack flows to launch an attack. The attack cost is linear in the number of attack flows selected by the attacker. Thus,

$$AC(n^a) = \alpha n^a, \qquad 0 \le n^a \le g.$$

Definition 4. The defense benefit $E^d(n^a, n^d)$ is the expected benefit generated by attack flows that the defender successfully scrubs when the attacker selects n^a attack flows and the defender selects n^d attack flows for scrubbing. The defense benefit is linear in the expected defense benefit. Let h_d denote the number of flows against which the defense succeeds. Then

$$E^a(n^a, n^d) = \mu h_d, \qquad 0 \le n^a \le g, \quad 0 \le n^d \le g.$$

Translation note[b].

Definition 5. The defense cost $DC(n^d)$ is the cost incurred when the defender selects n^d attack flows for scrubbing. The defense cost is linear in the number of attack flows selected for scrubbing by the defender. Thus,

$$DC(n^d) = \beta n^d, \qquad 0 \le n^d \le g.$$

The following theorems can be derived from these definitions.

Theorem 1. *Suppose that the attacker randomly selects n^a of g types of attack flows and the defender randomly selects n^d types of flows for scrubbing. The sum of the numbers of flows selected by the attacker over all attack–defense states is*

$$h_a = n^a * C_g^{n^a} * C_g^{n^d}.$$

Proof. The attacker has $C_g^{n^a}$ possible states, and the defender has $C_g^{n^d}$ possible states. Therefore, there are $C_g^{n^a} * C_g^{n^d}$ combinations of attack–defense states.

Let h_a^i, where $1 \le i \le C_g^{n^a} * C_g^{n^d}$, denote the number of attack flows sent by the attacker in the ith attack–defense state. The attacker selects the same number of flows, n^a, in every attack–defense state.

The total number of attack flows sent by the attacker is

$$h_a = h_a^1 + h_a^2 + \cdots + h_a^{C_g^{n^a} * C_g^{n^d}} = n^a * C_g^{n^a} * C_g^{n^d},$$

where $0 \le n^a \le g$ and $0 \le n^d \le g$. □

Theorem 2. *Suppose that the attacker randomly selects n^a of g types of attack flows and the defender randomly selects n^d types of flows for scrubbing. The total number of attack flows against which the defender succeeds over all attack–defense states is*

$$h_d = n^a * C_g^{n^a} * \left(C_g^{n^d} - C_{g-1}^{n^d}\right).$$

Proof. The defender can select n^d of g types of attack traffic for scrubbing and therefore has $C_g^{n^d}$ possible states. For any attack state, let Y be the set of attack flows sent by the attacker and X the set of flows against which the defender provides protection. For every $y \in Y$, the number of attack–defense states in which $y \notin X$ is $C_{g-1}^{n^d}$. Thus, the total number of successful defenses of flow y is $C_g^{n^d} - C_{g-1}^{n^d}$.

Because $|Y| = n^a$, there are n^a choices of $y \in Y$. Therefore, for attack set Y, the number of flows against which the defender succeeds is

$$n^a * \left(C_g^{n^d} - C_{g-1}^{n^d}\right).$$

Moreover, there are C_g^n choices for the attacker's attack-flow set Y. Consequently, the total number of attack flows against which the defender succeeds is

$$h_d = n * C_g^{n^d} * \left(C_g^{n^d} - C_{g-1}^{n^d}\right), \qquad 0 \le n^a \le g, \quad 0 \le n^d \le g.$$

□

Translation note[c].

If the attacker randomly selects n^a attack flows to launch an attack, the probability of each attack state is $p_a = 1/C_g^{n^a}$. If the defender randomly selects n^d attack flows for scrubbing, the probability of each defense state is $p_d = 1/C_g^{n^d}$.

Theorem 3. *Suppose that the attacker randomly selects n^a of g attack flows to launch an attack and the defender randomly selects n^d attack flows for scrubbing. The attacker's expected benefit is $\lambda * n^a * (g - n^d)/g$.*

Proof. Let h_a^i, where $1 \le i \le C_g^{n^a} * C_g^{n^d}$, denote the number of attack flows sent by the attacker in the ith attack–defense state, and let h_d^i, with the same range of i, denote the number of flows against which the defender succeeds in that state. In the ith attack–defense state, the number of attack flows against which the defense is unsuccessful is

$$h_k = h_a^i - h_d^i, \qquad 1 \le i \le C_g^{n^a} * C_g^{n^d}.$$

The attacker's benefit in this attack–defense state is therefore $p_a p_b \lambda (h_a^i - h_d^i)$.

Over all attack–defense states, the attacker's benefit is

$$\begin{aligned} E^a(n^a, n^d) = {} & p_a p_b \lambda (h_a^1 - h_d^1) + p_a p_b \lambda (h_a^2 - h_d^2) + \cdots \\ & + p_a p_b \lambda \left(h_a^{C_g^{n^a} * C_g^{n^d}} - h_d^{C_g^{n^a} * C_g^{n^d}} \right). \end{aligned}$$

Furthermore,

$$h_a^1 + h_a^2 + \cdots + h_a^{C_g^{n^a} * C_g^{n^d}} = \sum_{i=1}^{C_g^{n^a} * C_g^{n^d}} h_a^i = h_a,$$

$$h_d^1 + h_d^2 + \cdots + h_d^{C_g^{n^a} * C_g^{n^d}} = \sum_{i=1}^{C_g^{n^a} * C_g^{n^d}} h_d^i = h_d.$$

Therefore,

$$\begin{aligned} E^a(n^a, n^d) &= p_a p_b \lambda (s_a - s_d) \\ &= \lambda * \frac{1}{C_g^{n^a}} * \frac{1}{C_g^{n^d}} * \left(n^a * C_g^{n^a} * C_g^{n^d} - n^a * C_g^{n^d} * \left(C_g^{n^d} - C_{g-1}^{n^d} \right) \right) \\ &= \lambda * \frac{1}{C_g^{n^d}} * n^a * C_{g-1}^{n^d} \\ &= \lambda * n^a * (g - n^d)/g. \end{aligned}$$

□

Translation note[d].

Theorem 4. *Suppose that the attacker randomly selects n^a of g attack flows to launch an attack and the defender randomly selects n^d attack flows for scrubbing. The defender's expected benefit is $\mu * n^a * (n^d/g)$.*

Proof. Let h_d^i, where $1 \le i \le C_g^{n^a} * C_g^{n^d}$, denote the number of flows against which the defender succeeds in the ith attack–defense state. In this attack–defense state, the attacker's benefit is $p_a p_b \mu h_d^i$.

Over all attack–defense states, the defender's benefit is

$$E^d(n^a, n^d) = p_a p_b \mu h_d^1 + p_a p_b \mu h_d^2 + \cdots + p_a p_b \mu h_d^{C_g^{n^a} * C_g^{n^d}}.$$

Furthermore,

$$h_d^1 + h_d^2 + \cdots + h_d^{C_g^{n^a} * C_g^{n^d}} = \sum_{i=1}^{C_g^{n^a} * C_g^{n^d}} h_d^i = h_d.$$

Therefore,

$$\begin{aligned} E^d(n^a, n^d) &= p_a p_b \mu h_d \\ &= \mu * \frac{1}{C_g^{n^a}} * \frac{1}{C_g^{n^d}} * n^a * C_g^{n^a} * \left(C_g^{n^d} - C_{g-1}^{n^d} \right) \\ &= \mu * n^a * (n^d/g). \end{aligned}$$

□

Translation note[e].

Combining the above results gives the attacker's payoff function:

$$f^a = E^a(n^a, n^d) - AC(n^a) = \lambda * n^a * (g - n^d)/g - \alpha n^a.$$

The defender's payoff function is

$$f^d = E^d(n^a, n^d) - DC(n^d) = \mu * n^a * (n^d/g) - \beta n^d.$$

The attack and defense payoff matrices can be obtained from f^a and f^d and used to solve for the Nash equilibrium of TSG.

Definition 6. Pure-strategy Nash equilibrium. Given a traffic scrubbing game TSG = $\{N, S, D, U\}$, let s^a be the attacker's strategy and s^d the defender's strategy. A strategy profile (s^{a*}, s^{d*}) is a pure-strategy Nash equilibrium of the game if and only if it is optimal for both the attacker and the defender; that is, for all s^a and s^d,

$$u^a(s^{a*}, s^{d*}) \geq u^a(s^a, s^{d*}) \quad \text{and} \quad u^d(s^{a*}, s^{d*}) \geq u^d(s^{a*}, s^d).$$

In some cases, a pure-strategy Nash equilibrium of TSG may not exist. It is therefore necessary to consider a mixed-strategy Nash equilibrium of TSG.

Let p_i^a denote the probability that the attacker selects attack strategy s_i^a, and let p_j^d denote the probability that the defender selects defense strategy s_j^d. For all i, j, $s_i^a \in S^a$ and $s_j^d \in S^d$,

$$0 \leq p_i^a \leq 1, \qquad 0 \leq p_j^d \leq 1, \qquad \sum_{i=1}^{g} p_i^a = 1, \qquad \sum_{j=1}^{g} p_j^d = 1.$$

The mixed attack probabilities $P^a = (p_1^a, p_2^a, \ldots, p_g^a)$ represent the attacker's probabilistic selection from all possible attack strategies. The mixed defense strategy $P^d = (p_1^d, p_2^d, \ldots, p_g^d)$ represents the defender's probabilistic selection from all possible defense strategies.

Translation note[f].

Definition 7. Mixed-strategy Nash equilibrium. Given a traffic scrubbing game TSG = $\{N, S, D, U\}$, a strategy profile (P^{a*}, P^{d*}) is a mixed-strategy Nash equilibrium if and only if this mixed strategy is optimal for both the attacker and the defender; that is, for all P^a and P^d,

$$u^a(P^{a*}, P^{d*}) \geq u^a(P^a, P^{d*}) \quad \text{and} \quad u^d(P^{a*}, P) \geq u^d(P^{a*}, P^d).$$

Translation note[g].

To find the defender's optimal scrubbing strategy, we design the algorithms `psNE` and `msNE` based on the definitions of Nash equilibrium for TSG. They solve for pure-strategy and mixed-strategy Nash equilibria, respectively, as shown in Algorithms 1 and 2. Algorithm 1 first initializes the attack and defense strategy sets (lines 1–2). It then initializes the two parties' payoff functions from the game parameters (lines 3–4) and uses those functions to generate their payoff matrices. Finally, it uses the maxima in the payoff matrices to solve for a pure-strategy Nash equilibrium (lines 5–8). If an equilibrium exists, it returns that equilibrium; otherwise, it returns the empty set (lines 9–12). Algorithm 2 uses linear programming to solve for a mixed strategy of the game.

As shown in Algorithm 3, `tsgNE` provides a Nash equilibrium of TSG based on its pure-strategy and mixed-strategy Nash equilibria. If a pure-strategy Nash equilibrium exists, it returns that

equilibrium; otherwise, it returns a mixed-strategy Nash equilibrium.

Algorithm 1. Algorithm for finding a pure-strategy Nash equilibrium of TSG. `psNE()`
Input: $\alpha, \beta, \lambda, \mu$
Output: A pure-strategy Nash equilibrium

1. Initialize the attack strategy set $S^a = \{s_0^a, s_1^a, s_2^a, \ldots, s_g^a\}$;
2. Initialize the defense strategy set $S^d = \{s_0^d, s_1^d, s_2^d, \ldots, s_g^d\}$;
3. $f^a = \lambda * n^a * (g - n^d)/g - \alpha n^a$;
4. $f^d = \mu * n^a * (n^d/g) - \beta n^d$;
5. For every $s_i^a \in S^a$, $s_j^d \in S^d$, generate the attacker's payoff matrix U^a;
6. For every $s_i^a \in S^a$, $s_j^d \in S^d$, generate the defender's payoff matrix U^d;
7. Find the maximum $\max\{a_{ij}\}$ in each column of U^a; let I be the corresponding set of indices;
8. Find the maximum $\max\{a_{ij}\}$ in each row of U^d; let J be the corresponding set of indices;
9. **if** $I \cap J \neq \varnothing$
10. **return** s^{d*}; // A pure-strategy Nash equilibrium exists; return the equilibrium.
11. **else**
12. **return** $\varnothing$; // No pure-strategy Nash equilibrium exists; return the empty set.

Algorithm 2. Algorithm for finding a mixed-strategy Nash equilibrium of TSG. `msNE()`
Input: $\alpha, \beta, \lambda, \mu$
Output: A mixed-strategy Nash equilibrium

1. **maximize** v // Maximize matrix v.
2. **subject to**
3. **for all** $s_j \in S_j$;
4. $\sum_{i=0}^{g} p_i U(s_i, s_j) \geq v$;
5. $\sum_{i=0}^{g} p_i = 1,\ p_i \geq 0$;

Algorithm 3. Nash equilibrium algorithm for TSG. `tsgNE()`
Output: A Nash equilibrium of TSG

1. `ret` = `psNE()`;
2. **if** `ret` $\neq \varnothing$
3. **return** `ret`;
4. **else**
5. **return** `msNE()`;

4 Design and Implementation

This report designs DDSS, a P4-based dynamic traffic-scrubbing system for link-flooding DDoS attacks. Its architecture is shown in Figure 4. DDSS consists primarily of a control plane, a data plane, and a southbound interface, with P4Runtime serving as the southbound interface. The control plane comprises an SDN controller and four core modules. The attack sensing module monitors the

status of links throughout the network to identify links under attack. Based on the current attack–defense state, the strategy-generation module analyzes the link-flooding DDoS attack–defense game using game theory and computes the optimal scrubbing strategy. The flow-management module configures P4 switches according to the defense strategy and installs flow-scrubbing rules on them. The VNF-management module installs traffic-scrubbing VNFs in the data plane according to the type of attack traffic.

The data plane consists of P4 switches and traffic-scrubbing VNFs. A P4 switch forwards network traffic according to the flow-processing logic configured by the controller, detects and identifies attack flows, and scrubs attack traffic according to the scrubbing rules. For attack flows that cannot be scrubbed using the match-action model, the P4 switch sends them to a traffic-scrubbing VNF for processing. Traffic-scrubbing VNFs are implemented using DPDK and include a Kernel NIC Interface (KNI) virtual network interface component, a lockless ring (Ring), a memory pool (MemPool), and a traffic-scrubbing application. The application can implement customized attack-flow scrubbing tasks.

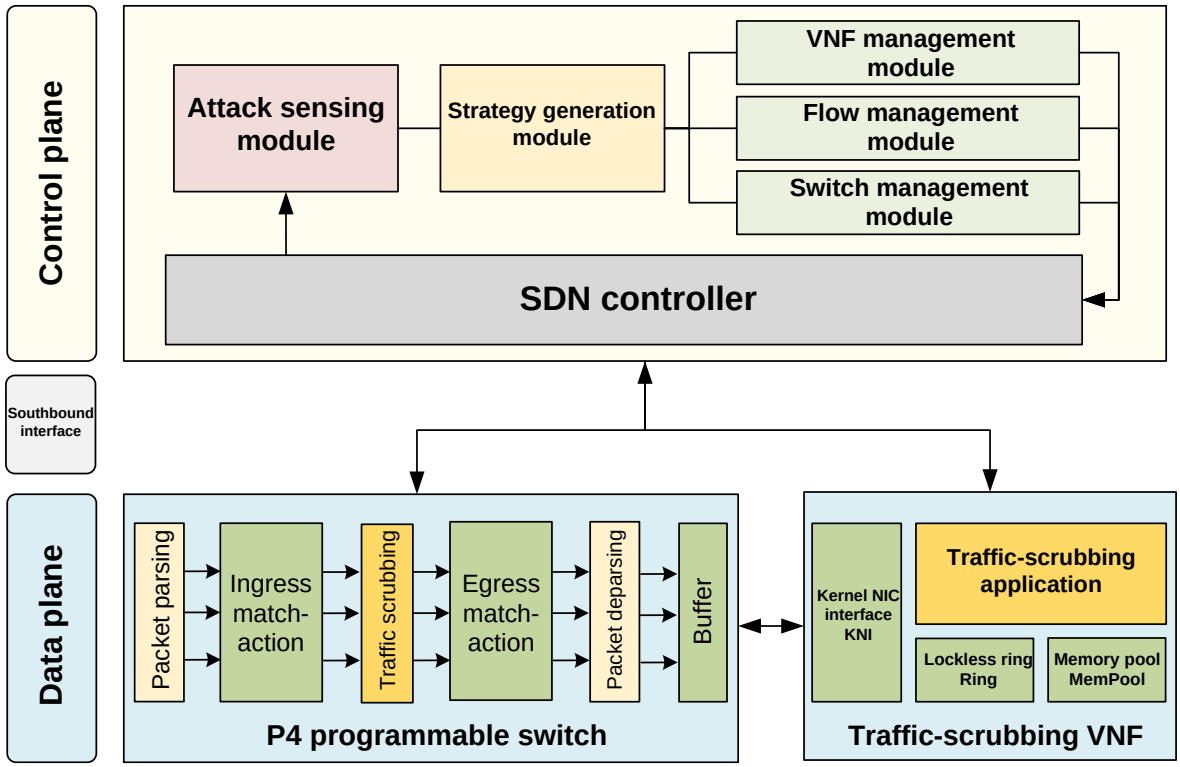


Figure 4: DDSS system architecture.

Translation note[h].

5 Experiments and Analysis

Experiments were conducted to evaluate the effectiveness and performance of DDSS. The experimental platform was a server with an Intel(R) Core(TM) i7-9750H CPU operating at 2.60 GHz and 32 GB of RAM, running Ubuntu 16.04 LTS. The emulated controller was developed using P4-Utils [21], the southbound interface used P4Runtime [22], and the emulated switches used BMv2 [23]. Mininet [24] was used to construct the network topology and emulate a real network environment. Figure 5 shows the experimental topology, which contains multiple hosts and switches. Each switch is connected to one scrubbing server. Link s6–s7 is the attacker's target link.

To emulate a link-flooding DDoS attack, the experiments used hping3 to emulate the attacker, with

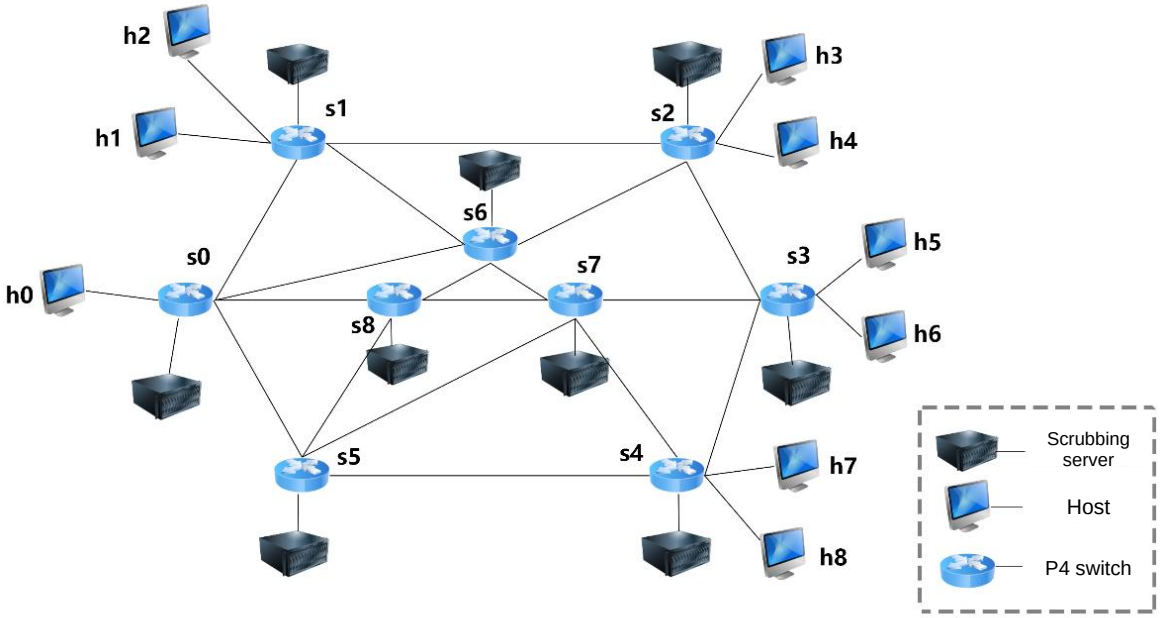


Figure 5: Experimental network topology.

hosts h0, h1, h2, and h3 serving as attack sources. These hosts could send seven potential attack flows and one legitimate flow into the network. Each flow had a rate of 100 Kb/s, and all flows traversed link s6–s7. Their routing paths are shown in Figure 6. The bandwidth of link s6–s7 was set to 300 Kb/s.

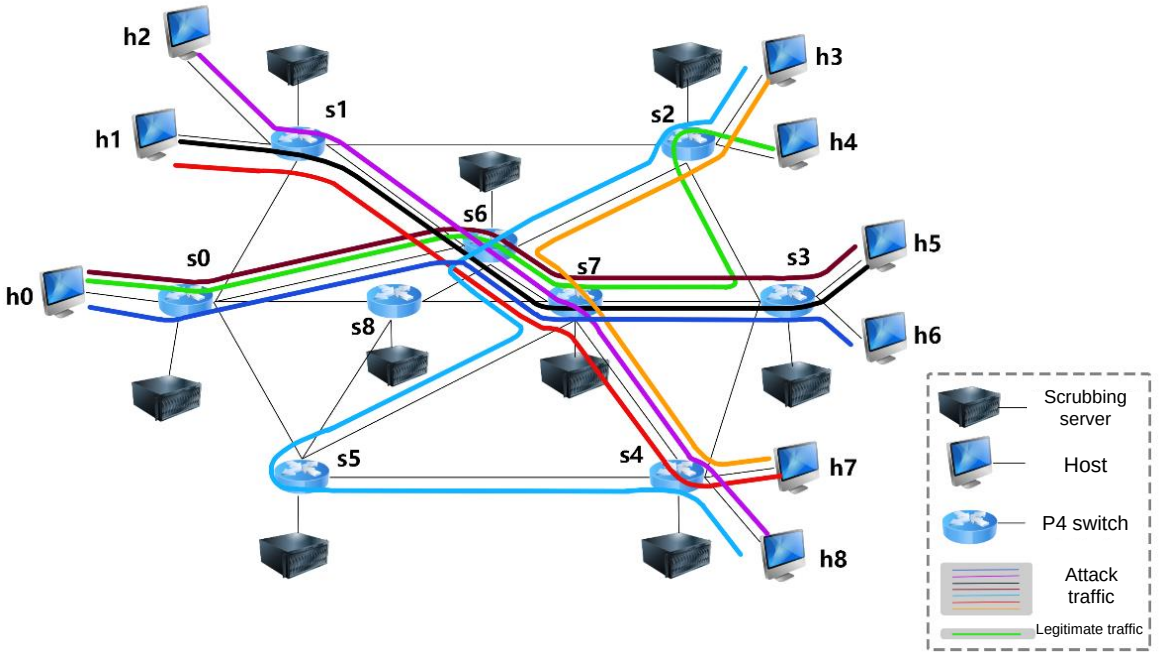


Figure 6: Traffic paths for the link-flooding DDoS attack.

5.1 Effectiveness of DDSS

Hosts h0, h1, h2, and h3 were used as attack sources. At each interval, five of the seven potential attack flows were selected at random and sent into the network, producing a total attack rate of 500 Kb/s. The attack-traffic rate on the target link, s6–s7, was measured for different numbers of scrubbing rules. ICMP flood traffic was used to emulate the attack. Such attack traffic can be scrubbed using the match-action model [18]; therefore, P4 switches can drop it directly without forwarding it to a traffic-scrubbing VNF. The defender used dynamic flow-scrubbing rules, and both the attack and defense intervals were 10 s. Figure 7 shows the experimental results. As the number of dynamic scrubbing rules increases, the attack-traffic rate on the target link decreases overall, although fluctuations remain. This occurs because both the attack and defense are dynamic, making the amount of traffic scrubbed uncertain.

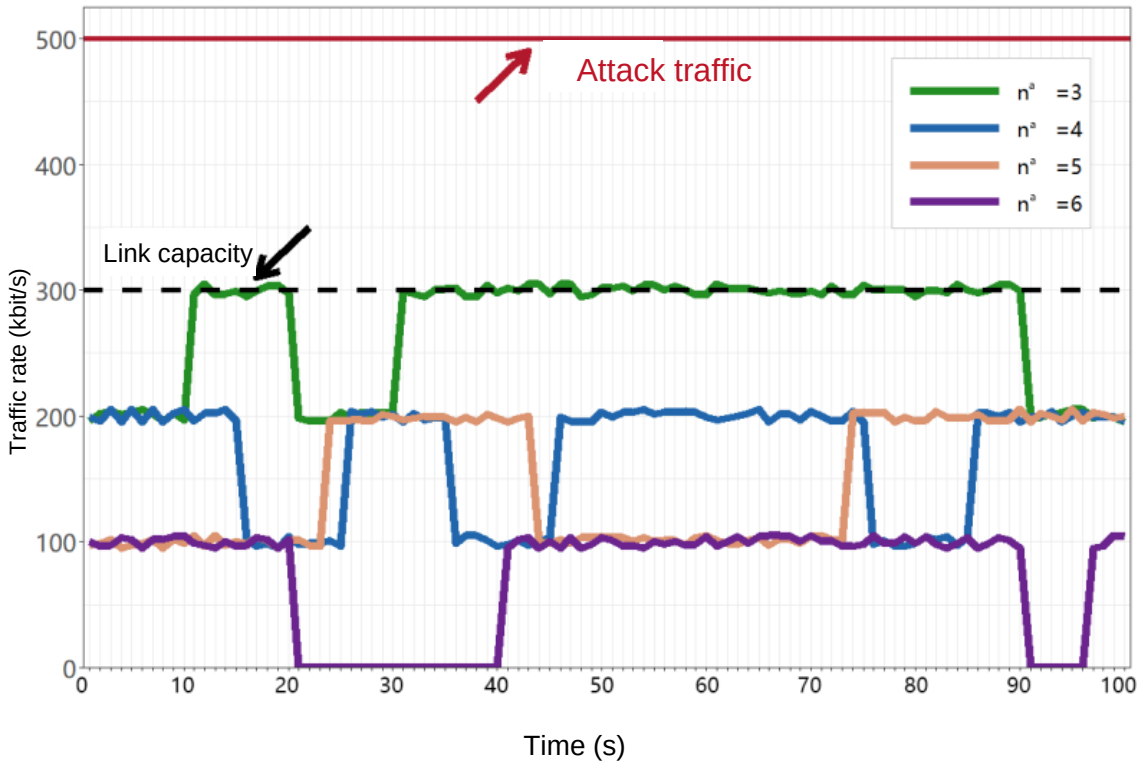


Figure 7: Attack-traffic rate on the target link over time under different scrubbing strategies during an ICMP flood attack.

To evaluate the ability of DDSS to scrub different traffic types, another experiment used SYN flood traffic to emulate the attack. This attack traffic cannot be scrubbed using the match-action model [18]. A P4 switch cannot complete the scrubbing by itself and must send the traffic to a traffic-scrubbing VNF for processing. Figure 8 shows the experimental results, which are similar to the results during a SYN flood attack. The experiments indicate that DDSS can effectively scrub both types of link-flooding DDoS attack traffic.

Translation note[i].

Translation note[j].

The Nash-equilibrium algorithm for the TSG game presented in this report was used to determine the optimal scrubbing strategy. The experimental parameters were set to $\alpha = 9$, $\beta = 8$, $\lambda = 13$, and $\mu = 13$. Both the attacker and the defender had the strategy set $\{0, 1, 2, 3, 4, 5, 6, 7\}$. Using the attacker and defender utility functions, f^a and f^d, respectively, gives the following attack–defense payoff matrix:

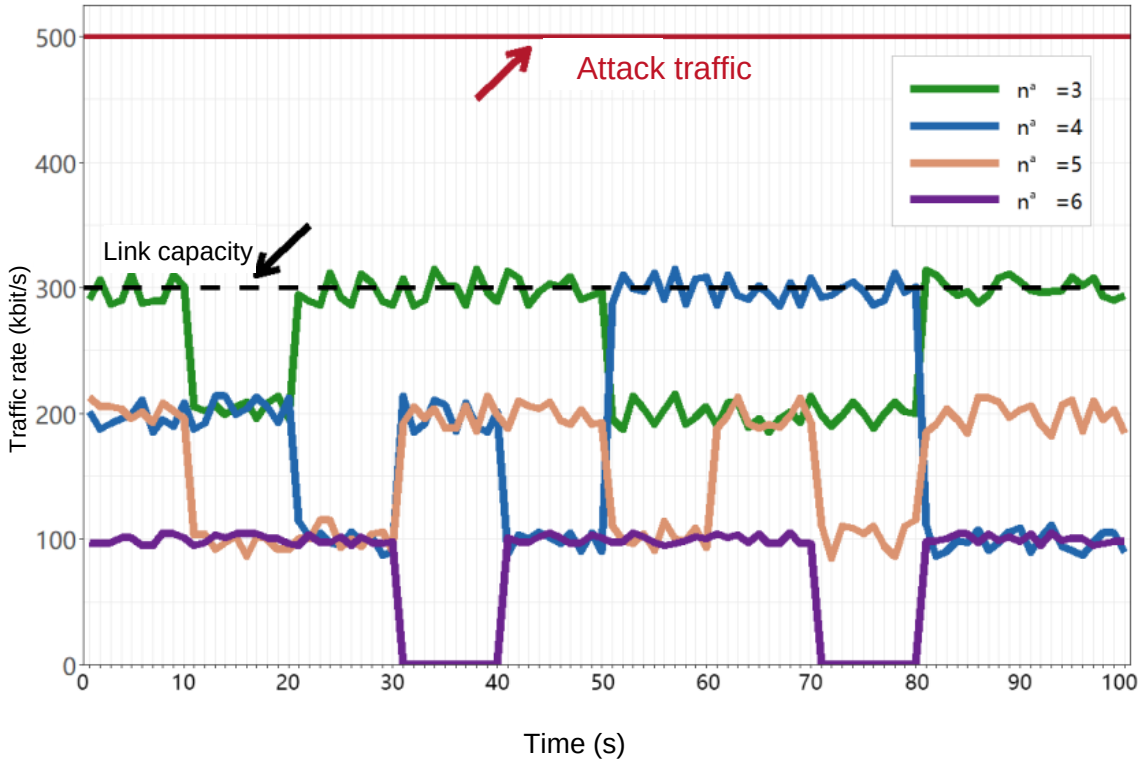


Figure 8: Attack-traffic rate on the target link over time under different scrubbing strategies during a SYN flood attack.

	s_d^0	s_d^1	s_d^2	s_d^3	s_d^4	s_d^5	s_d^6	s_d^7
s_a^0	$(0,0)$	$(0,-80)$	$(0,-160)$	$(0,-240)$	$(0,-320)$	$(0,-400)$	$(0,-480)$	$(0,-560)$
s_a^1	$(40,0)$	$(21,-61)$	$(2,-122)$	$(-15,-184)$	$(-34,-245)$	$(-52,-307)$	$(-71,-368)$	$(-90,-430)$
s_a^2	$(80,0)$	$(42,-42)$	$(5,-85)$	$(-31,-128)$	$(-68,-171)$	$(-105,-214)$	$(-142,-257)$	$(-180,-300)$
s_a^3	$(120,0)$	$(64,-24)$	$(8,-48)$	$(-47,-72)$	$(-102,-97)$	$(-158,-121)$	$(-214,-145)$	$(-270,-170)$
s_a^4	$(160,0)$	$(85,-5)$	$(11,-11)$	$(-62,-17)$	$(-137,-22)$	$(-211,-28)$	$(-285,-34)$	$(-360,-40)$
s_a^5	$(200,0)$	$(107,12)$	$(14,25)$	$(-78,38)$	$(-171,51)$	$(-264,64)$	$(-357,77)$	$(-450,90)$
s_a^6	$(240,0)$	$(128,31)$	$(17,62)$	$(-94,94)$	$(-205,125)$	$(-317,157)$	$(-428,188)$	$(-540,220)$
s_a^7	$(280,0)$	$(150,50)$	$(20,100)$	$(-110,150)$	$(-240,200)$	$(-370,250)$	$(-500,300)$	$(-630,350)$

Nash proved that every finite game has an equilibrium point [25]. Algorithm 3 yields the Nash equilibrium (P^{a*}, P^{d*}) of the TSG game, where the attacker's strategy is

$$P^{a*} = \left\{ \frac{5}{13}, 0, 0, 0, \frac{8}{13}, 0, 0, 0 \right\},$$

and the defender's strategy is

$$P^{d*} = \left\{ 0, 0, \frac{11}{13}, \frac{2}{13}, 0, 0, 0, 0 \right\}.$$

Thus, the defender's optimal traffic-scrubbing strategy is to select two flows for scrubbing with probability 11/13 and three flows with probability 2/13. This scrubbing strategy maximizes the attacker's utility.

Translation note[k].

5.2 Performance of DDSS

The experiments compared dynamic traffic scrubbing using DDSS with conventional static traffic scrubbing for link-flooding DDoS attack traffic. In each interval, the defender randomly selected

three of the seven potential attack flows for scrubbing, and network latency and available bandwidth were measured. For latency, ping measured the round-trip time (RTT) between h0 and h4 once per second, for a total of 100 measurements. For bandwidth, iperf measured the bandwidth between h0 and h4 once per second, also for a total of 100 measurements. Both the attack and defense intervals were 10 s. Figure 9 shows the average latency and average available bandwidth at different attack intensities.

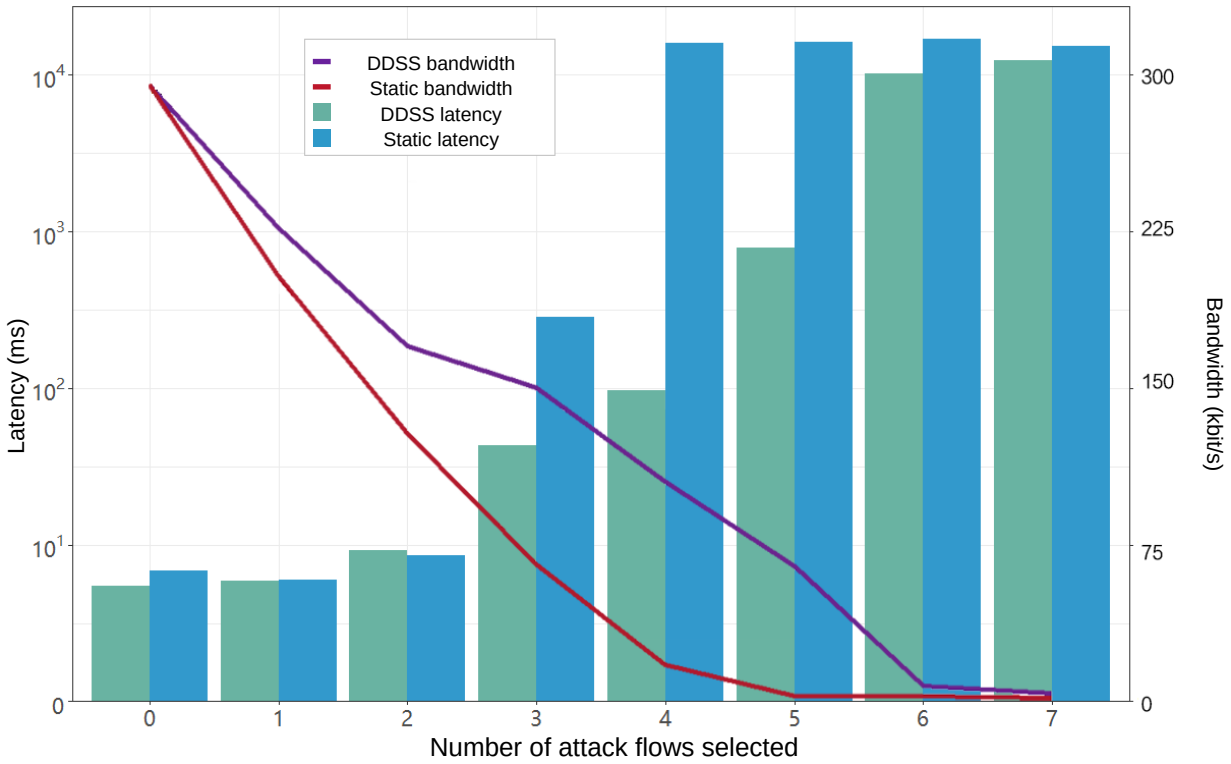


Figure 9: Average latency and available bandwidth of DDSS and static defense at different attack intensities. The horizontal axis is the number of attack flows selected by the attacker; bars show latency and lines show available bandwidth.

The experimental results show that DDSS and static defense have similar network latency and bandwidth when the attack intensity is either very low or very high. At a very low attack intensity, the attack traffic is insufficient to congest the network link and cannot produce an effective attack; the network therefore performs well with both DDSS and static scrubbing. At a very high attack intensity, neither DDSS nor static scrubbing can completely remove the attack traffic, and network performance is poor under both methods. In this case, however, the attacker uses substantial resources and incurs a high attack cost. When the number of attack flows is greater than two and less than six, DDSS produces lower network latency and higher available bandwidth on the target link. This is because a dynamic attacker can readily bypass static scrubbing rules to mount another effective attack. By contrast, the attacker has difficulty predicting the dynamic scrubbing strategy that DDSS will use in the next interval and therefore cannot mount a targeted link-flooding DDoS attack.

The experiments also compared the network latency of DDSS, Poseidon [18], and conventional P4 switches. The attacker launched an attack using a fixed set of five attack flows, and the RTT between h0 and h4 was measured. Switch s5 and its connected scrubbing server were used to emulate the traffic-scrubbing center of Poseidon. Figure 10 shows the results. In the absence of an attack, the network latencies of both Poseidon and DDSS are similar to that of the conventional P4 network. When attack traffic needs to be scrubbed, however, Poseidon exhibits a marked increase in network latency, whereas DDSS exhibits only a small increase. Poseidon must send a large volume of attack traffic to the scrubbing center for centralized processing and then forward the traffic to

the destination host. This lengthens the transmission path and causes congestion on other links in the network, thereby increasing network latency. DDSS scrubs attack traffic locally within the network, without increasing the network load or path length, and therefore introduces relatively little additional network latency.

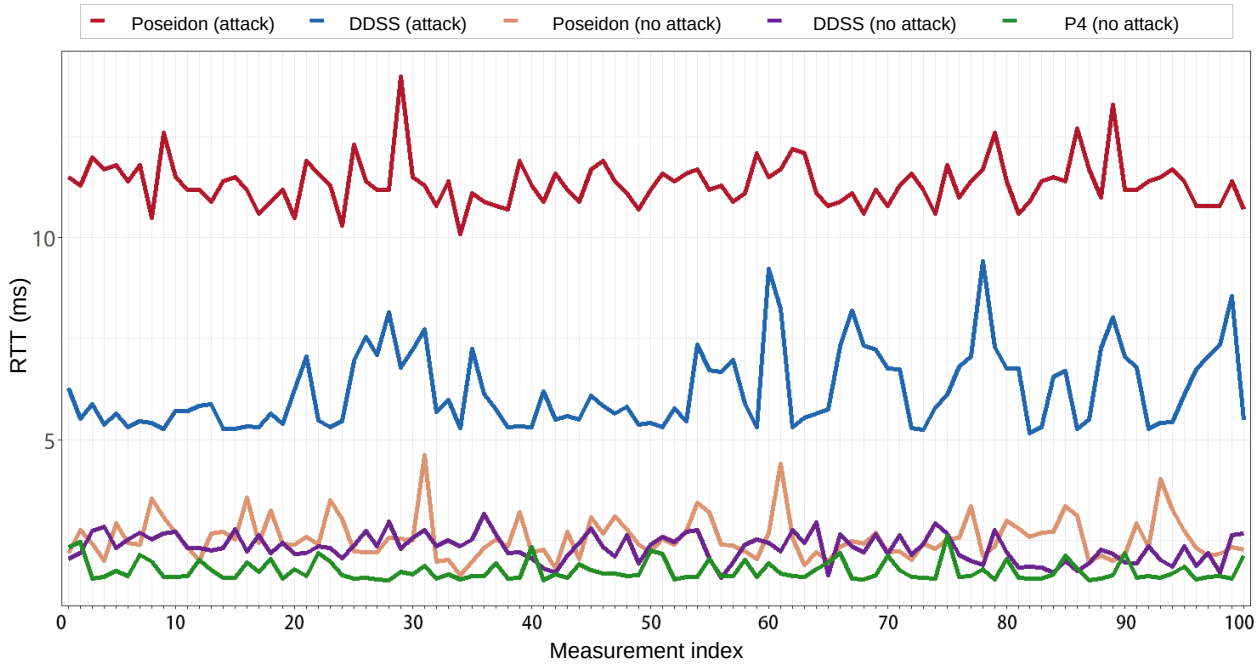


Figure 10: RTT comparison across 100 measurements: DDSS and Poseidon with and without an attack, and the conventional P4 network without an attack.

5.3 Overhead of DDSS

The experiments compared the controller CPU overhead of DDSS and static defense. The attacker periodically selected five attack flows at random to launch an attack, while the defender periodically selected four flows at random for scrubbing. The experiment ran for 200 s. Average controller CPU utilization was measured for different defense intervals, and the results are shown in Figure 11.

The figure shows that shorter defense intervals lead to higher average controller CPU utilization. This is because the controller must recalculate and install flow rules more frequently, consuming more CPU cycles. With a defense interval of 10 s, DDSS has approximately 30% higher CPU utilization than static defense. As the defense interval increases, the average CPU utilization of DDSS gradually decreases and approaches that of static defense. At a defense interval of 100 s, DDSS has approximately 0.08% higher CPU utilization than static defense. The experiment indicates that the CPU overhead of the proposed DDSS method is small.

6 Conclusion

This report introduces moving target defense (MTD) into traffic scrubbing and proposes DDSS, a dynamic traffic-scrubbing method for link-flooding DDoS attacks based on a programmable data plane. The method combines P4 switches with traffic-scrubbing servers to defend against multiple types of attack traffic and uses dynamic scrubbing rules to counter dynamic attackers. Link DDoS attack and defense are modeled as a static game of complete information, and the optimal scrubbing strategy is determined using Nash equilibrium. Experiments show that DDSS can effectively defend against dynamic link-flooding DDoS attacks involving multiple types of attack traffic with low overhead.

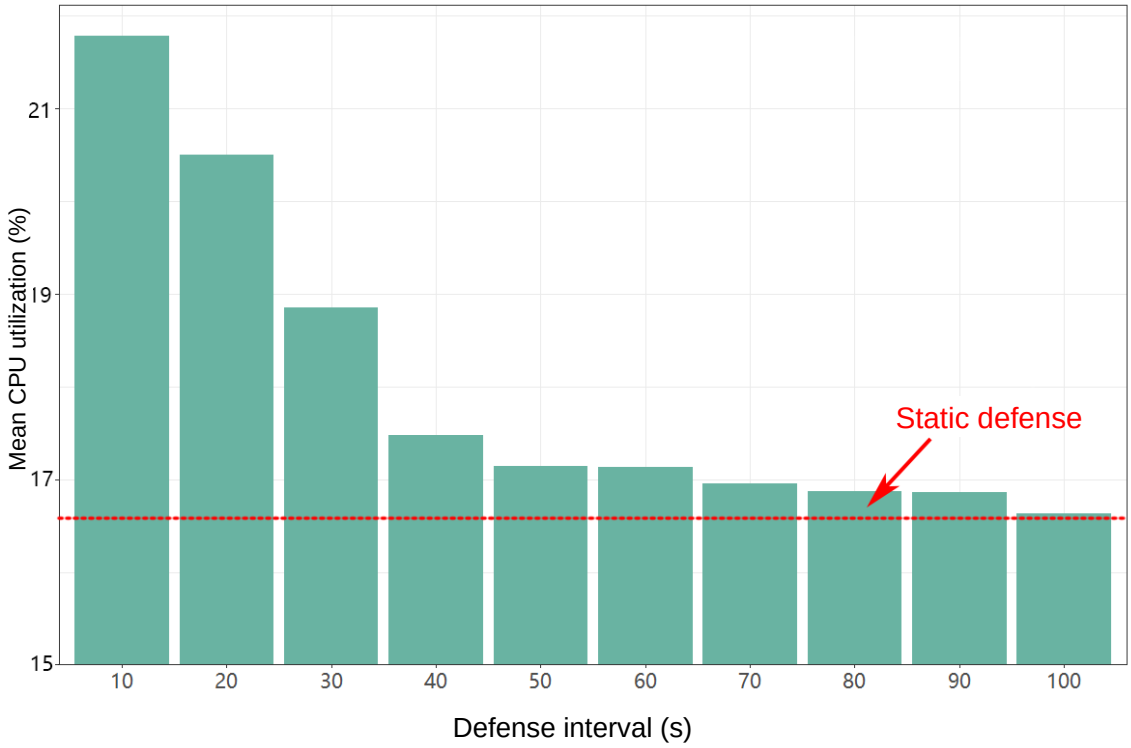


Figure 11: Average controller CPU utilization of DDSS for different defense intervals, compared with the static-defense baseline.

In this work, each traffic-scrubbing VNF is deployed as a basic unit on a traffic-scrubbing server. When the attack traffic changes, the VNF must be reinstalled, resulting in substantial deployment overhead. Future work will modularize VNFs for different types of attack traffic and investigate dynamically distributing these modular VNFs across multiple scrubbing servers in the network to reduce deployment overhead.

References

[1] R. Rasool, H. Wang, U. Ashraf, et al. A survey of link flooding attacks in software defined network ecosystems. *Journal of Network and Computer Applications*, 2020, 172: 102803.

[2] Guilin Cai, Baosheng Wang, Tianzuo Wang, et al. Research progress in moving target defense technology [in Chinese]. *Journal of Computer Research and Development*, 2016, 53(5): 968–987.

[3] P. Bosshart, D. Daly, G. Gibb, et al. P4: Programming protocol-independent packet processors [J]. *ACM SIGCOMM Computer Communication Review*, 2014, 44(3): 87–95.

[4] L. Zhou and H. Guo. Applying NFV/SDN in mitigating DDoS attacks. *TENCON 2017–2017 IEEE Region 10 Conference*. IEEE, 2017: 2061–2066.

[5] A. Studer and A. Perrig. The coremelt attack. In *2009 European Symposium on Research in Computer Security*. Springer, 2009.

[6] M. S. Kang, S. B. Lee, and V. D. Gligor. The Crossfire Attack. In *2013 IEEE Symposium on Security and Privacy*. IEEE, 2013.

[7] D. Gkounis, V. Kotronis, C. Liaskos, et al. On the interplay of link-flooding attacks and traffic engineering. *ACM SIGCOMM Computer Communication Review*, 2016, 46(2): 5–11.

[8] C. Liaskos, V. Kotronis, and X. Dimitropoulos. A novel framework for modeling and mitigating distributed link flooding attacks. *IEEE INFOCOM 2016–The 35th Annual IEEE International Conference on Computer Communications*. IEEE, 2016: 1–9.

[9] X. Ma and Y. Chen. DDoS detection method based on chaos analysis of network traffic entropy. *IEEE Communications Letters*, 2013, 18(1): 114–117.

[10] J. Chen, Y. Yang, K. Hu, et al. DAD-MCNN: DDoS attack detection via multi-channel CNN. *Proceedings of the 2019 11th International Conference on Machine Learning and Computing*, 2019: 484–488.

[11] L. Xue, X. Ma, X. Luo, et al. Linkscope: Toward detecting target link flooding attacks. *IEEE Transactions on Information Forensics and Security*, 2018, 13(10): 2423–2438.

[12] X. Xing, T. Luo, J. Li, et al. A defense mechanism against the DNS amplification attack in SDN. *2016 IEEE International Conference on Network Infrastructure and Digital Content (IC-NIDC)*, 2016: 28–33.

[13] J. Wang, R. Wen, J. Li, et al. Detecting and mitigating target link-flooding attacks using SDN. *IEEE Transactions on Dependable and Secure Computing*, 2018, 16(6): 944–956.

[14] L. Xie, Y. Ding, H. Yang, et al. Mitigating LFA through segment rerouting in IoT environment with traceroute flow abnormality detection. *Journal of Network and Computer Applications*, 2020, 164: 102690.

[15] S. B. Lee, M. S. Kang, and V. D. Gligor. CoDef: Collaborative defense against large-scale link-flooding attacks. 2013: 417–428.

[16] X. Ding, F. Xiao, M. Zhou, et al. Active link obfuscation to thwart link-flooding attacks for internet of things. *2020 IEEE 19th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*. IEEE, 2020: 217–224.

[17] S. K. Fayaz, Y. Tobioka, V. Sekar, et al. Bohatei: Flexible and Elastic DDoS Defense. *24th USENIX Security Symposium (USENIX Security 15)*, 2015: 817–832.

[18] M. Zhang, G. Li, S. Wang, et al. Poseidon: Mitigating volumetric DDoS attacks with programmable switches. *27th Network and Distributed System Security Symposium (NDSS 2020)*, 2020.

[19] F. Zhang, G. Jin, H. Zhang, et al. A new border filtering scheme against DDoS attacks. *2009 2nd International Conference on Power Electronics and Intelligent Transportation System (PEITS)*. IEEE, 2009, 1: 336–340.

[20] D. Sisodia, J. Li, and L. Jiao. In-network filtering of distributed denial-of-service traffic with near-optimal rule selection. *Proceedings of the 15th ACM Asia Conference on Computer and Communications Security*, 2020: 153–164.

[21] P4-utils. https://github.com/nsg-ethz/p4-utils. Accessed: August 2, 2022.

[22] N. McKeown, T. Sloane, and J. Wanderer. P4 runtime—Putting the control plane in charge of the forwarding plane. https://p4.org/api/p4-runtime-putting-the-control-plane-in-charge-of-the-forwarding-plane.html. Accessed: June 22, 2018.

[23] P4 Consortium. (2018a). BMv2: Designing your own switch target with BMv2. http://www.bmv2.org. Accessed: June 13, 2018.

[24] B. Lantz, N. Handigol, B. Heller, and V. Jeyakumar. Introduction to Mininet. Mininet Project, 2015.

[25] J. Nash. Non-cooperative games. *Annals of Mathematics*, 1951: 286–295.

Translation Notes

[a]Definition 1 reproduces the source tuple $\{N, S, D, U\}$ and the first element s_0^a of S^d. The listed strategy spaces are S^a and S^d; Algorithm 1 instead begins S^d with s_0^d.

[b]Definition 4 uses E^a in the displayed relation although it defines E^d. Its statement that defense benefit is linear in expected defense benefit is also reproduced as written.

[c]The final paragraph of the proof of Theorem 2 uses C_g^n, n, and $C_g^{n^d}$ exactly as in the source; these differ from $C_g^{n^a}$, n^a, and $C_g^{n^a}$ in the theorem statement.

[d]The proof of Theorem 3 uses p_b although p_d was defined, and its final derivation uses $s_a - s_d$ although the preceding sums define $h_a - h_d$. The factor $C_g^{n^d}$ multiplying the subtracted term is also reproduced as printed.

[e]The first paragraph of the proof of Theorem 4 says "attacker's benefit," although this theorem concerns the defender. The proof again uses the undefined p_b in place of the previously defined defense-state probability p_d.

[f]The mixed-strategy sums and vectors begin at index 1 in the source, although Definition 1 and Algorithms 1–2 include the strategy with index 0.

[g]The second inequality in Definition 7 uses an unqualified P in its left-hand side, as printed in the source, instead of the P^{d*} used in the strategy profile.

[h]The source text identifies four core control-plane modules, while Figure 4 also shows a switch-management module. Both the text and the diagram are preserved.

[i]The source compares the SYN flood results with "the results during a SYN flood attack." This self-comparison is retained; the intended comparison may have been with the preceding ICMP flood experiment.

[j]The legends in Figures 7 and 8 use $n^a = 3, 4, 5, 6$, although the surrounding text describes varying the number of scrubbing rules. The original legend notation is retained.

[k]The final sentence says "the attacker's utility" in the source, despite describing the defender's optimal scrubbing strategy. This wording, the reported payoff matrix, and the reported equilibrium probabilities are reproduced without correction.